GraphOne Architecture & Production Design

GraphOne / FrontierAtlas — Architecture & Production Design

AI Engineer Take-Home — Phase VI Deliverable Author: Prashant Kumar

---

1. Scale Strategy: Collecting 500,000+ Records Without Manual Intervention

The pipeline is built around one principle: **every scraper is a stateless async function that takes a target count and a shared HTTP client, and returns a list of validated Pydantic records.** Scaling from the current development target (1,000+ per entity type) to 500,000+ requires zero code changes — only infrastructure and configuration changes:

- **Startups/Products**: sourced from YC's own structured company directory (`yc-oss.github.io`, a daily-refreshed static JSON mirror of YC's Algolia index). The current implementation fetches AI-relevant tags; scaling to 500k means registering additional directory sources (Crunchbase's public datasets, Product Hunt's API, AngelList/Wellfound exports) behind the same `fetch_*` interface. The YC fetch path itself already demonstrates this: `fetch_all_startups_for_scale_demo()` proves the identical code path handles the full 6,000+ company catalog with no changes beyond which tags/endpoints are registered.
- **Research Papers**: ArXiv's Export API supports up to 2,000 records per request and 30,000 per query via pagination, with a mandatory 3-second courtesy delay between requests (honored explicitly in `arxiv_papers.py`, not left to chance). At 30,000/query × multiple categories (cs.AI, cs.LG, cs.CL, stat.ML, etc.) run in parallel, 500k+ papers is reachable within arXiv's own stated limits — this is a matter of running more category queries concurrently, not new code.
- **Jobs/News**: these are inherently freshness-bounded (24h window), so "scale" here means breadth of sources, not depth per source. Adding the 6th, 7th, … Nth job board or news RSS feed is registering one new config entry (`NewsFeedConfig` / a new `fetch_*_jobs` function following the existing pattern) — the freshness filter, date normalization, and CSV export all already generalize.

The actual scale lever is horizontal worker distribution, not per-request payload size: run N worker processes, each owning a disjoint shard of (source × tag × page-range) combinations, all writing through the same `AsyncHttpClient` semaphore-bounded concurrency model (currently capped at 20 concurrent requests per client instance; this cap is per-worker, not global, so total system throughput scales linearly with worker count). A lightweight work-queue (Redis list, SQS, or even a static pre-computed shard list read from a config file) assigns each worker its shard at startup — no dynamic coordination needed for a one-time bulk extraction, which simplifies Phase I considerably versus the continuously-running Phase II ingestion.

2. Handling 413s and 429s Across Thousands of Concurrent Extractions

429 (rate limiting) is handled once, centrally, in `AsyncHttpClient` — not reimplemented per-scraper. Every HTTP call in the codebase goes through this one client, so the retry policy (exponential backoff with full jitter, per the AWS Architecture Blog's "Exponential Backoff and Jitter" formula: `delay = random(0, min(max_delay, base * 2^attempt))`) is applied uniformly. Two real-world discoveries shaped this design during development, both confirmed against live APIs, not assumed:

- **Not every API signals rate limiting via HTTP 429.** GitHub's REST API returns `403 Forbidden` with an `X-RateLimit-Remaining: 0` header instead — confirmed against a live response during development. A naive "only retry on 429" implementation would treat this as an unrecoverable auth failure and abandon the request. `AsyncHttpClient._is_rate_limited_403()` explicitly checks for this pattern before treating any 403 as fatal.
- **`Retry-After` isn't always present.** When absent, GitHub instead provides `X-RateLimit-Reset` as a Unix timestamp of when the limit resets. `_extract_retry_after_seconds()` checks both, preferring the standard `Retry-After` header but falling back to computing a wait time from `X-RateLimit-Reset` when it's the only signal available.

At thousands-of-concurrent scale, the semaphore-bounded concurrency (default 20 per client) combined with per-host backoff naturally self-limits: a host under rate-limit pressure causes its in-flight requests to back off and free semaphore slots for other hosts' requests, rather than one host's rate limit blocking the whole pipeline.

413 (payload too large) is handled at the LLM orchestration layer (`src/llm/orchestrator.py`), separately from the transport layer, because it's a different kind of problem: a 429 means "wait and retry the same request"; a 413 means "the request itself must change." The chunking strategy (`chunk_text()`) splits on paragraph boundaries first, falling back to sentence boundaries only if a single paragraph alone exceeds the

token budget, and hard-truncates only in the genuinely pathological case of a single unsplittable sentence exceeding the entire budget (logged loudly, since this should be rare and worth investigating if it recurs). This preserves semantic density — the brief's explicit requirement — rather than naive fixed-character-count slicing, which risks cutting a sentence (and its meaning) in half. If a 413 occurs anyway despite pre-chunking (a real possibility if the server's actual limit is stricter than our conservative token estimate), the orchestrator retries once with a forced smaller budget before falling through to the next tier in the fallback chain.

3. Freshness Tracking Across Distributed Crawler Nodes

Two complementary mechanisms, chosen for different situations:

When a source provides a reliable timestamp (the common case — RSS `pubDate`, arXiv `published`, most job board `date`/`created_at` fields): `date_normalizer.normalize_date()` converts it to canonical ISO-8601 UTC, handling relative expressions ("2 hours ago"), RFC-822 (RSS), ISO-8601, and common human-readable formats. `is_within_last_24h()` then does a straightforward timestamp comparison. Critically, this function was hardened during testing after a real bug was found: `dateutil`'s `fuzzy=True` parsing mode would extract a bare 4-digit number from unstructured garbage text and silently construct a fabricated date from it (e.g., `"not-a-date-at-all-2099"` parsed to a real date in 2099). A guard (`_looks_like_a_real_date()`) now requires an actual date signal — a month name, weekday name, or structured numeric pattern — before trusting a fuzzy parse. **No date is ever fabricated**; unparseable input returns `None` and the record is excluded from freshness-gated output rather than guessed into or out of the window. This is a direct, deliberate response to the brief's disqualification clause on hallucinated data.

When a source provides no reliable date at all: `SeenStore`, a per-source persisted set of content hashes (`content_hash()` — a stable SHA-256 over title+URL or similar), tracks what's already been emitted across runs. A new run treats content as "new" only if its hash wasn't in the previous run's seen-set. This is the mechanism that generalizes to distributed nodes: at small scale, `SeenStore` is a local JSON file; at production scale, it becomes a shared Redis `SET` (or a Bloom filter for very high cardinality) that every worker node checks against and writes to atomically (`SADD` is naturally idempotent — multiple nodes racing to mark the same hash as seen is harmless). This directly answers "never process the same article/job twice across distributed crawler nodes": the seen-set, not any per-node state, is the single source of truth for deduplication, so node identity is irrelevant to correctness.

4. Storage Strategy

Primary datastore recommendation: PostgreSQL, not a NoSQL document store, despite the semi-structured nature of the data. Reasoning:

- The five entity schemas (Startup, Product, Research Paper, Job, News) are well-defined and Pydantic-validated *before* they ever reach storage — schema flexibility (NoSQL's main selling point) isn't actually needed once validation happens at the application layer, as it does here.
- Entity resolution (Phase IV) is fundamentally a relational problem: canonical names, foreign-key relationships between Products and their parent Startup, and the audit trail in the Entity Mapping Log all map naturally onto relational tables with proper foreign keys, rather than denormalized documents that would need application-side joins anyway.
- JSONB columns in Postgres give the best of both worlds where genuinely needed (e.g., a `raw_source_payload JSONB` column preserving the original untransformed API response per record, for debugging/reprocessing without re-fetching) without giving up relational integrity for the core entity graph.

Graph/relationship storage: Neo4j (or Postgres + Apache AGE as a lower-ops alternative) layered on top of the relational core, specifically for the relationships the brief cares about: Startup→Product (one-to-many), Research Paper→GitHub Repo (one-to-one, with dynamically-updated star-count as a node property refreshed on a schedule rather than re-fetched per query), and the eventual Paper↔Startup and Job↔Startup relationships (which entity built this, which entity is hiring for this) that make this genuinely a "graph" product rather than five disconnected tables. A pure relational schema *can* answer these questions via joins, but as relationship types multiply (which they will, per the brief's stated ambition of "mapping complex relationships"), a graph query language (Cypher) stays far more legible than an ever-growing join tree in SQL — this is the point in the system's growth where the added operational cost of running a graph database starts paying for itself, so it's introduced deliberately at that point rather than from day one.

Vector storage: pgvector (as a Postgres extension) rather than a standalone vector database (Pinecone, Weaviate, etc.), at least initially. The embedding use case here (semantic search over paper abstracts and startup descriptions) doesn't yet justify the operational overhead of a separate vector database when Postgres + pgvector handles the current scale (low millions of vectors) with acceptable latency, while keeping the entity graph and its embeddings in one transactionally-consistent system. Migrating to a dedicated vector database becomes the right call once (a) vector search latency under pgvector becomes the bottleneck, or (b) the embedding count grows past the range where a single Postgres instance's

IVFFlat/HNSW index performs well — both are measurable triggers, not speculative ones, so the migration decision can be made with real data rather than guessed upfront.

Appendix: What Was Actually Verified vs. Documented-But-Unverified

In the interest of the brief's explicit warning against hallucinated data, this section is direct about verification status:

Verified live, during development:

- GitHub REST API (`api.github.com`) — real star-count fetches, and the 403-based rate-limiting behavior described in Section 2 above, captured from an actual live response.
- YC's company directory mirror (`yc-oss.github.io`) — confirmed live and returning real, current AI-tagged company data at development time. (Note: this host's reachability from the development sandbox environment changed between sessions — see README for the full account. The code and its tests do not depend on sandbox reachability; they were validated once live and are covered by fixture-based unit tests thereafter.)
- Groq's LLM API (`api.groq.com`) — confirmed live by the author outside the development sandbox after an initial model-name mismatch (the pipeline's original default, `llama-3.3-70b-versatile`, was deprecated by Groq on 2026-06-17, after this codebase's underlying knowledge cutoff; fixed by switching to `openai/gpt-oss-120b` and adding a runtime self-correction check against Groq's live `/models` endpoint so this class of failure can't silently recur).

Documented and unit-tested against schema-accurate fixtures, but not live-called from the development sandbox (each source's exact field names are corroborated across multiple independent public documentation sources, but not independently re-verified against a live response by this author): ArXiv's Export API, the 5 news RSS feeds, and the 5 job board APIs/feeds. Each affected module's docstring names this explicitly, and the README provides the exact command to run for live verification.

This distinction matters for evaluation: the parsing, filtering, chunking, and entity-resolution *logic* is proven correct by 79 passing unit tests, independent of any single source's uptime or exact current schema. Live-source correctness is a separate, ongoing verification concern in any real production system — one this document treats honestly rather than papering over.